

실습일지 — 지도 기반 노선 타당성 분석 시뮬레이터

컴퓨터시스템공학과 3학년 2학기 · 202245079 신선호

심화응용프로젝트 · 2026년 10월 8일

1. 프로젝트 소개

지도에 노선을 그리면 건설비와 예상 이용객, B/C(편익비용비)를 계산해 주고, 그 구간에 맞는 교통수단과 정차역을 추천하는 웹 서비스를 만들고 있습니다. 예비타당성조사(KDI PIMAC) 방법론을 참고해서 학부 수준으로 축소한 것입니다.

보여주고 싶은 건 "이 구간은 BRT로 충분한데 지하철을 지으면 B/C가 0.4로 떨어진다"를 숫자로 확인하는 것입니다. BRT와 경전철은 다니는 구간이 비슷한데 건설비는 10배 넘게 차이가 납니다.

제안서에 쓴 기능 8개는 지난 기간에 다 만들어 놓았습니다. 그래서 이번에는 아직 정확하지 않은 부분을 고치고, 실제로 써 볼 만한 상태인지 확인하는 데 시간을 썼습니다.

2. 이번에 한 일

커밋은 15개를 했습니다. 성격별로 나누면 다음과 같습니다.

분류	내용
모델 정확도	단면 환산계수를 연장에 따라 바뀌게 수정, 지하비율 추정치 위험도 측정, 도심거리 기준점 수정
데이터 확장	지방 도시철도 좌표 10개 노선 271억 수집, 인구 집계구 52,584 → 70,840개
보안	소스에 들어 있던 JWT 기본 비밀키 제거
화면 버그	지도 크기, 비교표 강조, 오류 메시지 위치, 결과 패널 폭, 경유점 표시, 회원가입 안내

화면 버그가 6개나 나온 게 의외였습니다. 빌드도 되고 API도 정상인데 막상 화면을 열어 보면 틀린 것들이었습니다.

3. 단면 환산계수를 연장의 함수로 바꾼 것

어떤 수단을 쓸 수 있는지 판정하려면 출퇴근 시간에 가장 붐비는 구간을 몇 명이 지나가는지 알아야 합니다. 이걸 역 승하차 인원에서 환산하는데, 그 계수를 0.2441이라는 고정값으로 쓰고 있었습니다. 문제는 이 값을 서울 30~60km짜리 노선에서 구해 놓고, 정작 이 서비스에서 그리는 5~35km 노선에 그대로 쓰고 있었다는 점입니다.

생각해 보면 사람들이 노선 전체를 타고 가는 게 아니라 일부 구간만 타니까, 노선이 길수록 한 지점을 지나가는 비율은 낮아집니다. 그러면 이 계수는 연장에 반비례해야 맞습니다.

첨두율부터 고쳐야 했습니다

처음 상관관계를 봤더니 $r=0.775$ 정도밖에 안 나왔습니다. 확인해 보니 첨두율(출퇴근 시간에 몰리는 비율)을 전 노선 공통값 하나로 쓰고 있어서, 노선마다 다른 혼잡 패턴이 환산계수에 섞여 들어가 있었습니다. 일별 시간대별 자료를 다시 받아 노선별로 첨두율을 구했더니 0.1035~0.1298로 꽤 차이가 났고, 이걸 분리하니 상관이 $r=0.865$ 로 올라갔습니다.

모델 선택

표본이 4개뿐이라 적합이 잘 되는 걸 고르면 과적합이 됩니다. 그래서 하나씩 빼고 맞춰 보는 교차검증(LOO)으로 비교했습니다.

모델	적합 오차	교차검증 오차
----	-------	---------

상수 0.2441 (기존)	0.0573	0.0784
$K = a / \text{연장 (채택)}$	0.0328	0.0437
$K = a + b/\text{연장}$	0.0329	0.0737

계수가 두 개인 모델은 적합 오차는 같은데 교차검증이 더 나빠졌습니다. 표본 4개에 계수 2개는 무리라는 뜻으로 보고 계수 하나짜리를 골랐습니다. 최종적으로 $K = \min(0.395, 10.06 / \text{연장km})$ 을 씁니다. 0.395는 모든 통행이 가장 붐비는 구간을 지날 때의 이론값이라 그보다 커질 수 없고, 덕분에 짧은 노선에서 값이 터무니없이 커지는 것도 막힙니다.

다만 한계가 있습니다. 표본이 30.6~59.8km 네 개인데 정작 서비스에서 그리는 노선은 5~35km라, 25.5km 아래는 전부 상한에 걸립니다. 표본 밖을 외삽하는 셈이라 보고서에 적어 두었습니다.

4. 지방 노선 좌표를 모으면서 겪은 일

"이미 전철이 다니는 길에 또 짓는 안"을 걸러내는 기능이 수도권에서만 동작하고 있었습니다. 부산에 노선을 그려도 아무 경고가 뜨지 않았다는 뜻입니다.

전국 역 정보를 주는 공공데이터포털이나 국가철도공단 API는 서비스키가 있어야 해서 쓸 수 없었습니다. 결국 키 없이 쓸 수 있는 OpenStreetMap(OSM)을 쓰기로 했는데, OSM은 누구나 편집할 수 있는 자료라 그대로 믿기가 어려웠습니다. 이 프로젝트에서는 출처가 불확실한 자료를 쓰지 않기로 했기 때문에 더 고민이 됐습니다.

국토교통부 역수와 대조하는 검증을 넣음

그래서 수집한 역 개수를 국토교통부 도시철도 운영현황의 역수와 비교해서, 2개 넘게 차이가 나면 그 노선은 저장하지 않도록 만들었습니다. 이 검증 덕분에 세 번이나 잘못된 데이터를 걸렸습니다.

시도	증상	원인
1차	10개 노선 전부 역수 미달 (99역 / 실제 271역)	OSM의 노선 정보에 묶여 있는 건 이름 없는 정차위치 점과 승강장이고, 정작 이름이 붙은 역 지점은 거기 들어 있지 않았습니다
2차	광주가 통째로 빠짐	응답이 1개만 와도 "0개는 아니니까" 하고 넘어가 버렸습니다
3차	대구 1호선이 1개 많음	코레일 대구역과 지하철 대구역이 101m 거리라 둘 다 잡혔습니다

세 경우 다 예러가 난 게 아니라 정상 응답으로 왔습니다. 검증을 안 넣었으면 그냥 통과했을 것이고, 특히 첫 번째는 3분의 1밖에 안 되는 데이터가 그대로 저장될 뻔했습니다.

서버가 불안정해서 고생한 부분

OSM 공개 서버가 504 오류를 자주 내서 처음에는 미리 서버를 같이 썼습니다. 그런데 미러는 응답은 빠르는데 데이터가 비어 있는 걸 정상(200 OK)으로 돌려줬습니다. 같은 질의로 부산을 조회했을 때 본 서버는 8개, 미러 두 곳은 0개가 나왔습니다. 결국 미러를 다 빼고 본 서버만 쓰되, 오류가 나면 기다렸다가 다시 요청하도록 했습니다. 대신 노선별로 결과를 저장해 두어서 중간에 끊겨도 이어서 받을 수 있게 만들었습니다.

최종적으로 10개 노선 전부 국토교통부 역수와 정확히 일치했습니다(40/43/17/14/21/35/29/30/20/22, 총 271역).

5. 보안 문제를 발견한 것

.env 파일에 JWT 비밀키를 채워 넣는 간단한 작업을 하다가 더 큰 문제를 발견했습니다. application.yml에 47바이트짜리 기본 비밀키가 하드코딩되어 있었는데, 이 파일은 깃허브 공개 저장소에 올라가 있었습니다. 환경변수로 키를 지정하지 않고 배포하면 누구나 볼 수 있는 키로 토큰에 서명하게 되고, 그러면 아무 사용자 ID나 관리자 권한을 넣은 토큰을 만들 수 있습니다.

값만 채우고 넘어갔으면 이 문제는 그대로 남았을 것입니다. 기본값 자체를 지워서, 키가 없으면 서버가 아예 안 뜨도록 바꿨습니다. 다만 그렇게만 해 두니 라이브러리가 "key byte array is 104 bits"라고만 알려줘서 원인을 알기 어려웠습니다. 키가 없는지, 짧은지, 환경변수가 안 들어왔는지 구분해서 메시지를 내도록 했습니다.

이것과 별개로, 예전에 서울 열린데이터광장 API 키를 .env.example에 적어 둔 채 공개 저장소에 올린 적이 있습니다. 파일은 고쳤지만 커밋 기록에 남아 있어서 키를 재발급받는 것 말고는 방법이 없습니다. 아직 처리하지 못한 부분입니다.

6. 어떻게 확인했는지

크게 세 가지 방법으로 확인했습니다.

먼저 자동 테스트입니다. 백엔드 26개에서 29개로, ETL 48개에서 53개로 늘렸습니다. 새로 넣은 건 이번에 실제로 겪은 문제를 그대로 넣은 것들입니다. 예를 들어 "OSM에서 19역만 받아온 경우는 탈락, 40역을 받은 경우는 통과"를 테스트로 박아 두었습니다.

두 번째는 실제로 실행해 보는 것입니다. 부산, 대구, 광주, 대전 각 노선을 따라 그려서 전부 100%로 걸리는지, 엉뚱하게 직선으로 가로지르면 안 걸리는지, 기존에 되던 서울 2호선은 그대로 동작하는지 확인했습니다. 적재된 노선이 39개 784역에서 49개 1,055역으로 늘었습니다.

세 번째가 화면을 직접 열어 보는 것인데, 이게 제일 많은 버그를 찾아냈습니다. 지도 크기가 안 맞아서 빈 곳을 클릭하면 좌표가 엉뚱하게 찍히던 버그, 노선이 하나뿐인데 비교표 전체가 강조되던 문제, 관리자 화면에서 오류 메시지가 화면 밖에 그려지던 문제가 다 여기서 나왔습니다. 빌드나 API 테스트로는 전혀 안 보이는 것들이었습니다.

잘못 판단한 것

데이터가 다 로딩되기 전에 화면을 보고 "3D가 깨졌다", "관리자 표가 비었다"고 판단할 뻔한 적이 두 번 있었습니다. 둘 다 몇 초 기다리면 정상이었습니다. 그 뒤로는 로딩이 끝난 다음에 보기로 했습니다.

부산 2호선에 직선을 긋고 "이건 걸려야 하는데 왜 안 걸리지" 했던 것도 틀린 판단이었습니다. 2호선이 경성대, 광안리 쪽으로 크게 휘는 노선이라 직선은 실제 선형과 다른 길이었습니다. 경유점을 찍어서 선형을 따라가니 87%로 제대로 걸렸습니다.

7. 마지막에 발견한 문제 두 가지

화면을 확인하다가 지방은 B/C가 아예 안 나온다는 걸 알았습니다. 인구 데이터를 수도권만 수집해 뒀기 때문이었습니다. 정차역 이름도 동 이름 대신 1, 2, 3으로 나왔습니다(역 이름을 집계구의 행정동명에서 가져오기 때문이다). 중복 투자 경고는 되는데 정작 B/C는 못 내놓는 반쪽짜리였던 셈입니다.

인구 수집 범위를 지방까지 넓혔더니(집계구 52,584 → 70,840개) 수요 모델의 R^2 가 0.551에서 0.555로 바뀌었습니다. 지방 데이터만 추가했는데 수도권 회귀가 달라질 이유가 없어서 이상하다고 생각하고 찾아봤더니, 환승 노선 수 15개가 바뀌어 있었습니다.

```
transfers = stations.groupby('station_name').line_name.nunique()
```

역 이름만으로 묶고 있었습니다. 그래서 지방 데이터가 들어오자 서울 시청역이 부산 시청역, 대전 시청역과 한 덩어리가 되면서 환승 노선 수가 2에서 4로 늘었습니다. 400km 떨어진 역을 환승역으로 센 것입니다. 하필 환승 노선 수가 수요 모델에서 영향이 제일 큰 변수(계수 1.08)라 계수가 바로 흔들렸습니다. 이름이 같으면서 1km 안에 있는 노선만 세도록 고치니 R^2 가 0.551로 돌아왔습니다.

같은 식으로 하나를 더 찾았습니다. 도심까지의 거리를 재는 기준점이 서울시청 하나로 고정되어 있어서, 부산 역이 326km로 계산됐습니다. 회귀 표본이 0.1~88.7km 범위인데 그 3.7배를 벗어난 값이라, 도심거리 항목 때문에만 예측값이 0.02배로 줄어들었습니다. 부산 B/C 0.019는 숫자는 나오지만 아무 의미가 없는 값이었습니다. 부산 사람이 서울로 출퇴근하는 게 아니니 가장 가까운 광역시 도심까지를 재도록 고쳤습니다.

부산 1호선 서면~부산대	고치기 전	고친 후
예상 이용객	2,202명/일	96,480명/일
B/C	0.019	0.82
정차역 이름	1, 2, 3	부전1동, 양정2동, 연산4동

둘 다 "바뀔 이유가 없는데 값이 바뀌었다"는 점이 걸려서 찾은 것입니다. R^2 가 0.551에서 0.555로 올라간 건 오히려 좋아진 것처럼 보여서 그냥 지나칠 수도 있었습니다.

8. 아직 못 한 것

과제	막힌 이유
BRT 표정속도	공식 자료를 6곳 찾아봤는데 다 없었습니다. 세종시 시간표가 이미지 파일이라 자동으로 읽을 수 없습니다
GTX급 수단 분리	구조를 바꿔야 해서 방향을 먼저 정해야 합니다
복선전철 수요 하한	상행/하행을 구분한 승하차 자료가 없습니다 (3곳 확인)
공법 변수 추가	저심도나 기존선 활용 여부에 대한 자료가 없습니다

어디까지 찾아봤고 왜 안 됐는지는 docs/research-log.md에 적어 두었습니다. 나중에 같은 걸 또 뒤지지 않으려고 만든 문서입니다.

9. 느낀 점

이번에 고친 문제들 중에 에러를 내면서 죽은 건 거의 없었습니다. OSM 미러는 빈 데이터를 200 OK로 보내줬고, 환승 버그는 R^2 를 오히려 올려서 좋아진 것처럼 보였고, JWT 기본키는 서버가 잘 돌아가기 때문에 문제인 줄도 몰랐습니다. 에러는 알아서 눈에 띄지만 이런 건 내가 기준을 정해 두지 않으면 계속 모른 채로 지나간다는 걸 알았습니다. 국토교통부 역수라는 외부 기준이 없었으면 3분의 1짜리 데이터가 그냥 들어갔을 것입니다.

지하비율 데이터 40건이 전부 추정치라서 "회귀에서 제일 중요한 변수인데 추정치라 위험하다"고만 적어 두고 있었는데, 정작 얼마나 위험한지는 재 본 적이 없었습니다. 일부러 오차를 넣고 200번 다시 돌려 보니 예측값이 $\pm 7\%$ 안에서 움직였습니다. 추정이 틀리면 계수가 그만큼 보정되기 때문이었습니다. 막연히 불안해하던 걸 숫자로 바꾸니 보고서에 쓸 수 있는 내용이 됐습니다.

화면은 직접 열어 봐야 한다는 것도 배웠습니다. 테스트가 다 통과해도 실제로 클릭해 보면 좌표가 어긋나거나 메시지가 화면 밖에 그려지는 일이 있었습니다. 반대로 내가 잘못 예상한 경우도 있었으니, 결과가 예상과 다를 때 코드부터 의심할 게 아니라 내 예상이 맞는지부터 확인해야겠습니다.

마지막으로, 작업하면서 알게 된 것들을 CLAUDE.md에 계속 적어 둔 게 도움이 많이 됐습니다. 다만 "백엔드는 DB_PORT로 맞춘다"고만 적어 놓고 정작 .env에는 그 항목이 없어서, 적힌 대로 해도 무조건 실패하는 상태였던 적이 있습니다. 그때 나온 Access denied 메시지는 비밀번호 문제처럼 보였는데 사실은 다른 DB에 붙은 것이었습니다. 그 뒤로는 증상과 실제 원인을 같이 적어 두고 있습니다.